

Programação em Python

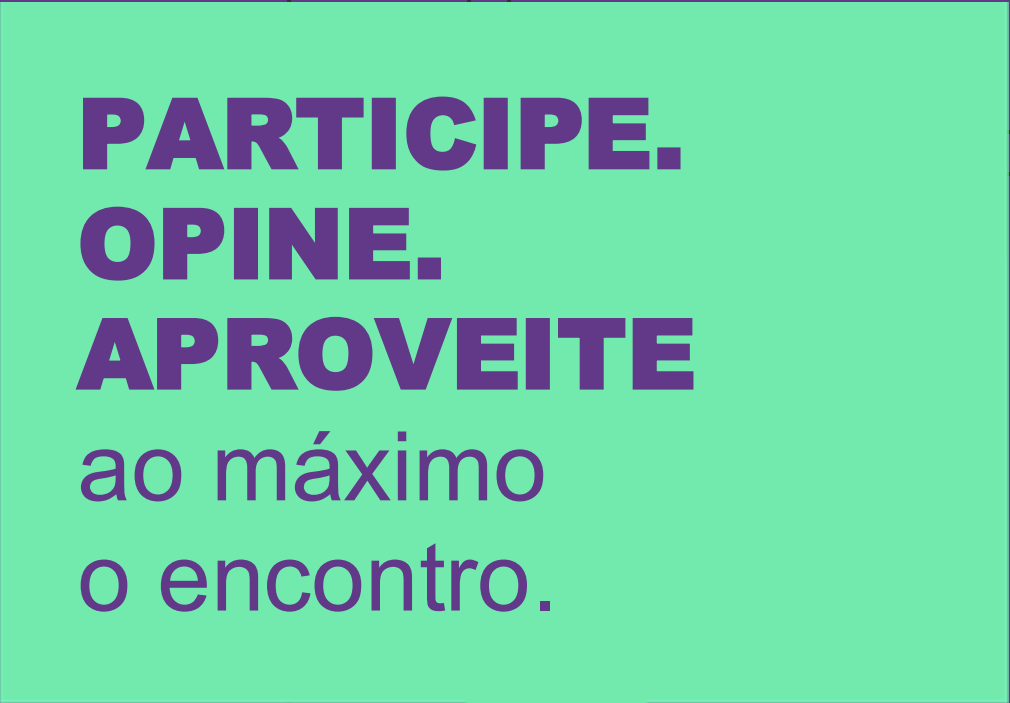
Desenvolver Aplicações Back- End para Web com Python



Bem-vindos à Aula: Desenvolver Aplicações **Back-End para web com python**

O que faremos hoje:

- Entender o que é POO
- Criar classes
- Criar objetos
- Utilizar atributos
- Utilizar métodos
- Utilizar o `__init__`
- Criar pequenos sistemas utilizando objetos



**PARTICIPE.
OPINE.
APROVEITE**
ao máximo
o encontro.

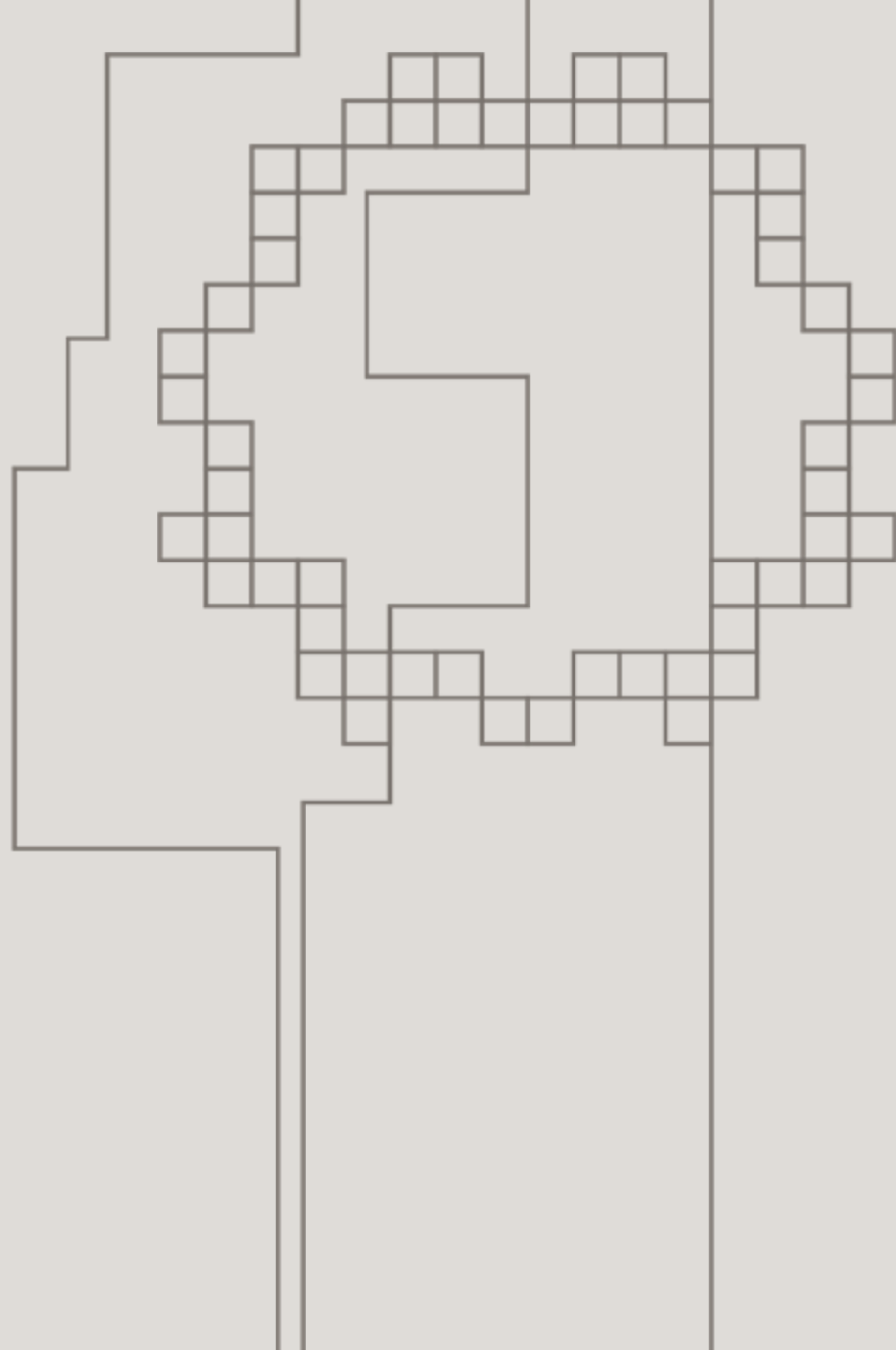
Estamos em um ambiente
heterogêneo e diverso,
onde sempre há respeito.

Então, lembre-se de que:

**NÃO EXISTE
RESPOSTA ERRADA,
PERGUNTA BOBA**
ou que não valha a pena.

Para não termos interrupções e em respeito aos colegas,
pedimos que não faça uso de celular durante a aula e que
o coloque no modo silencioso.

O que um carro
possui?





Você aperta um botão - o
elevador executa uma tarefa.



- cor
- modelo
- velocidade
- portas

E o carro pode:

- acelerar
- frear
- ligar



Reflexão

Na Programação Orientada a Objetos:

- características → atributos
- ações → métodos

Comparação Visual

Vida Real	Python
carro real	objeto
planta do carro	classe
acelerar	método
cor	atributo

Primeiro Exemplo

Python

```
class Carro:  
  
    def buzinar(self):  
        print("Biiiii!")
```

Criando Objeto

Python

```
carro1 = Carro()  
  
carro1.buzinar()
```



Primeiro desafio



Desafio

Crie uma classe chamada `Pessoa`.

Ela deverá possuir um método chamado `falar`.

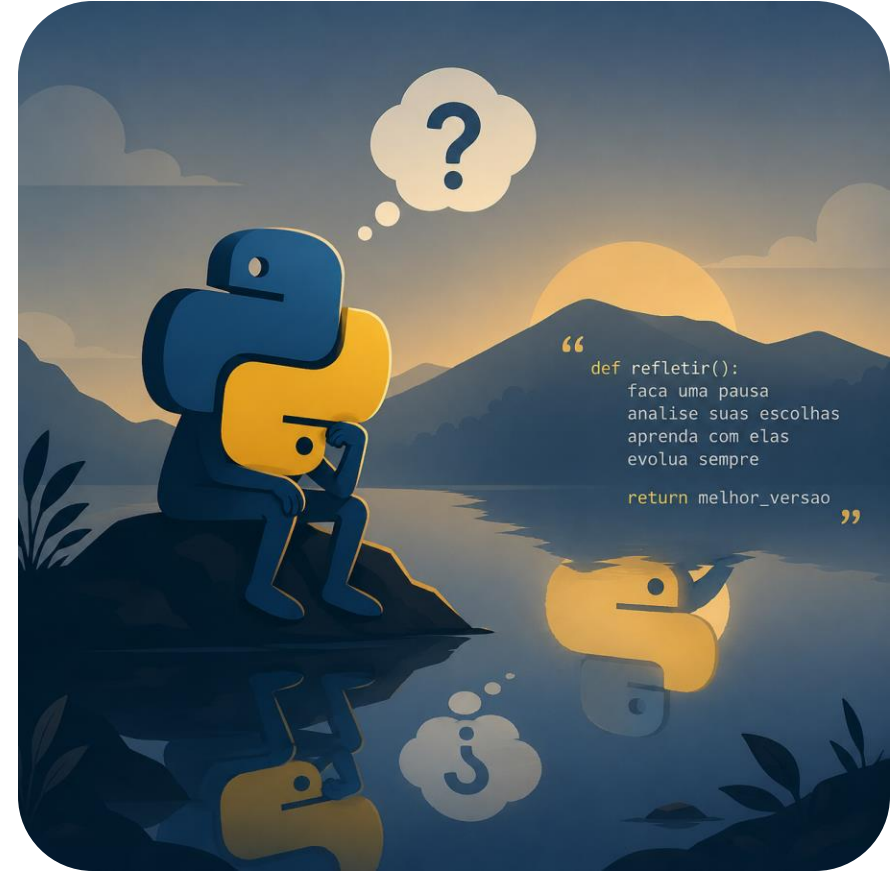


Reflexão

Estrutura de Classe

`</> Python`

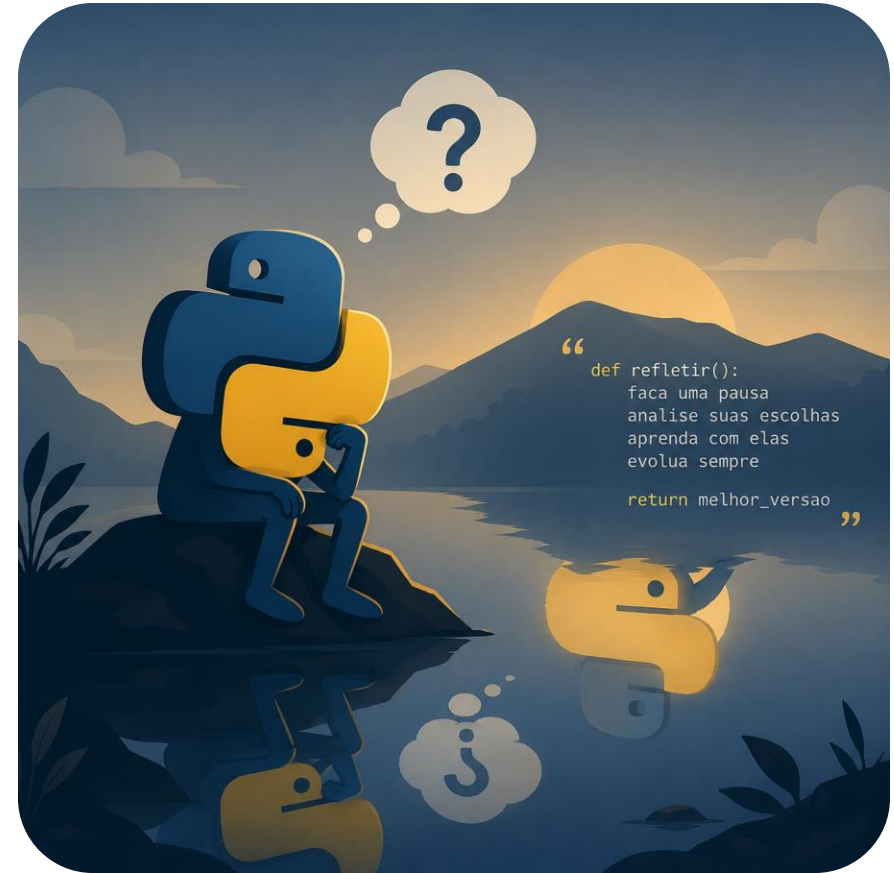
```
class Pessoa:
```



Método

⌞ Python

```
def falar(self):
```



```
“  
def refletir():  
    faça uma pausa  
    analise suas escolhas  
    aprenda com elas  
    evolua sempre  
    return melhor_versao  
”
```


Código explicado:

Código Completo

Python

```
class Pessoa:

    def falar(self):
        print("Olá!")
```

Criando Objeto

Python

```
pessoa1 = Pessoa()

pessoa1.falar()
```

Conceitos:

Explicação do `self`

O `self` representa:

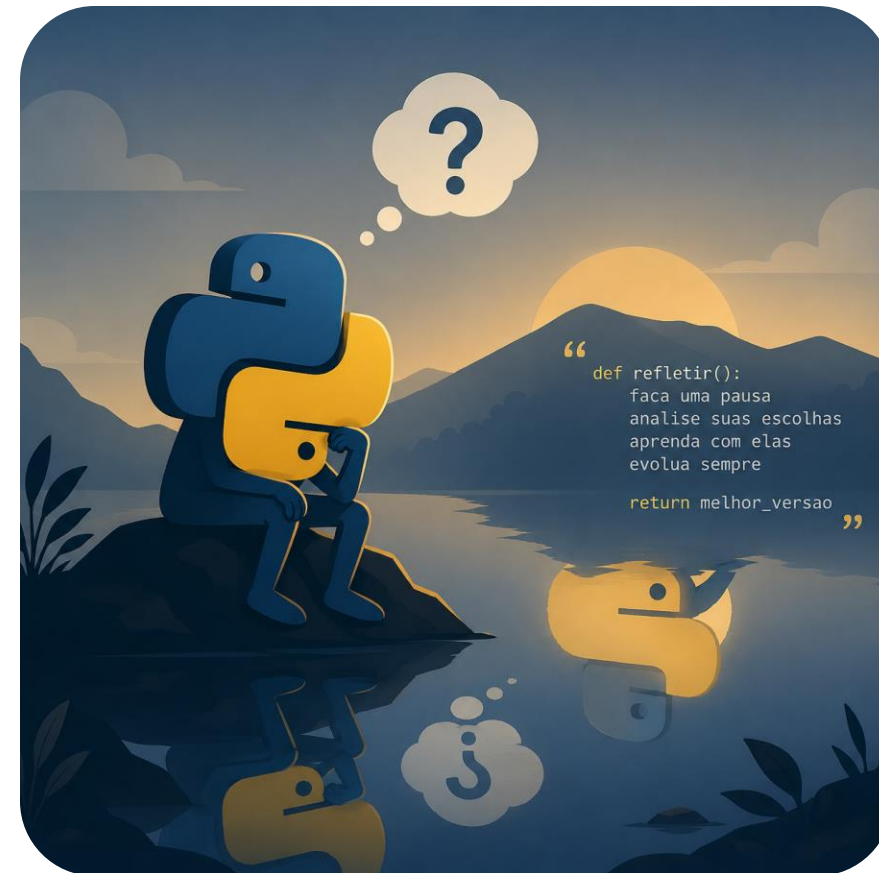
- o próprio objeto
- a própria pessoa criada

A classe é como uma fábrica.

O objeto é o produto criado.

Exemplo:

- Classe → Planta de uma casa
- Objeto → Casa construída





Outros desafios



Desafios

1. Classe `Animal`
2. Classe `Computador`
3. Classe `Celular`

Cada uma deverá possuir um método simples.



`</>` Python

```
class Animal:
```

```
    def emitir_som(self):  
        print("Som do animal")
```





Novo desafio



Criar uma classe `Aluno` com:

- nome
- idade

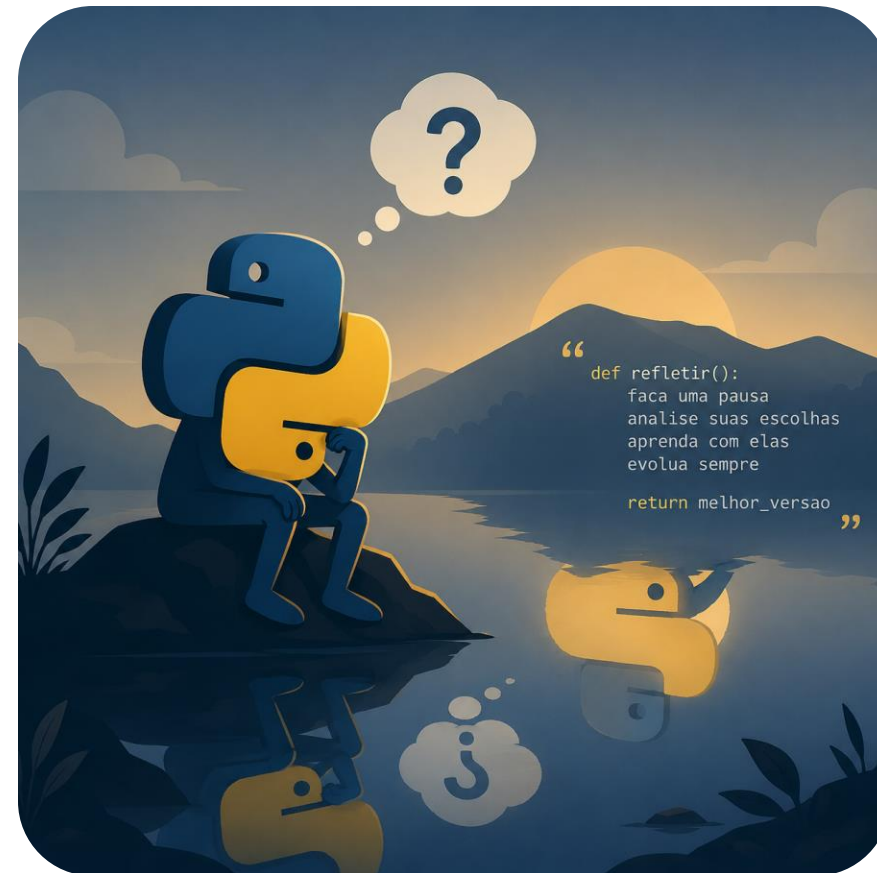




Reflexão

O que são atributos?

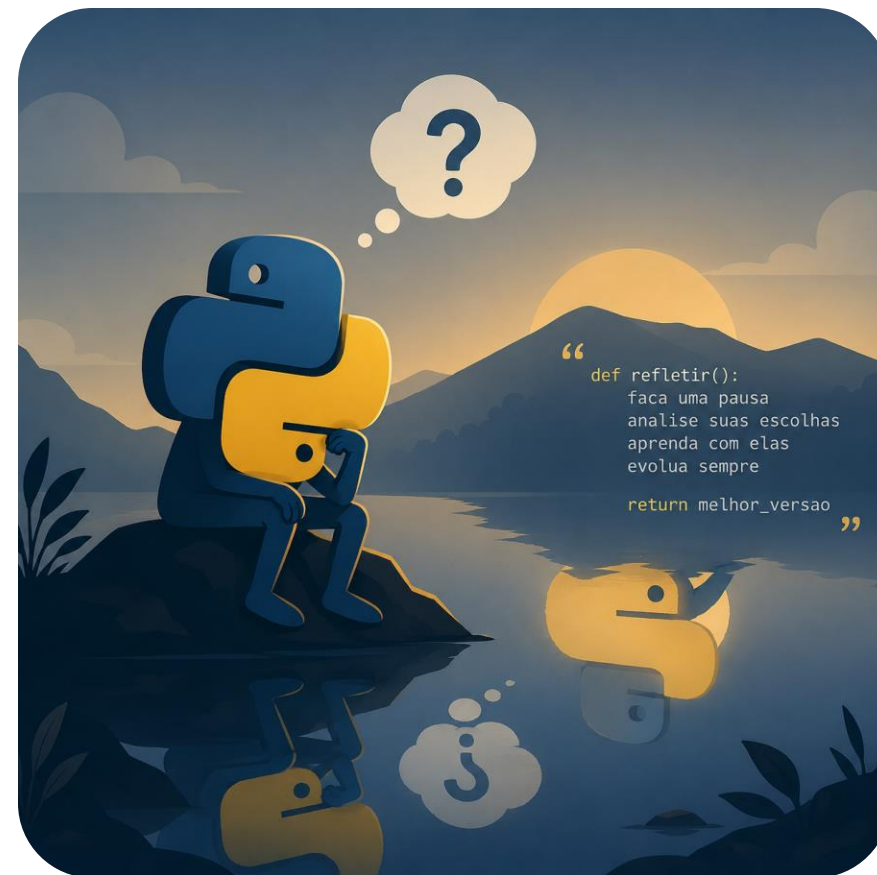
Atributos são características do objeto.



Um aluno possui:

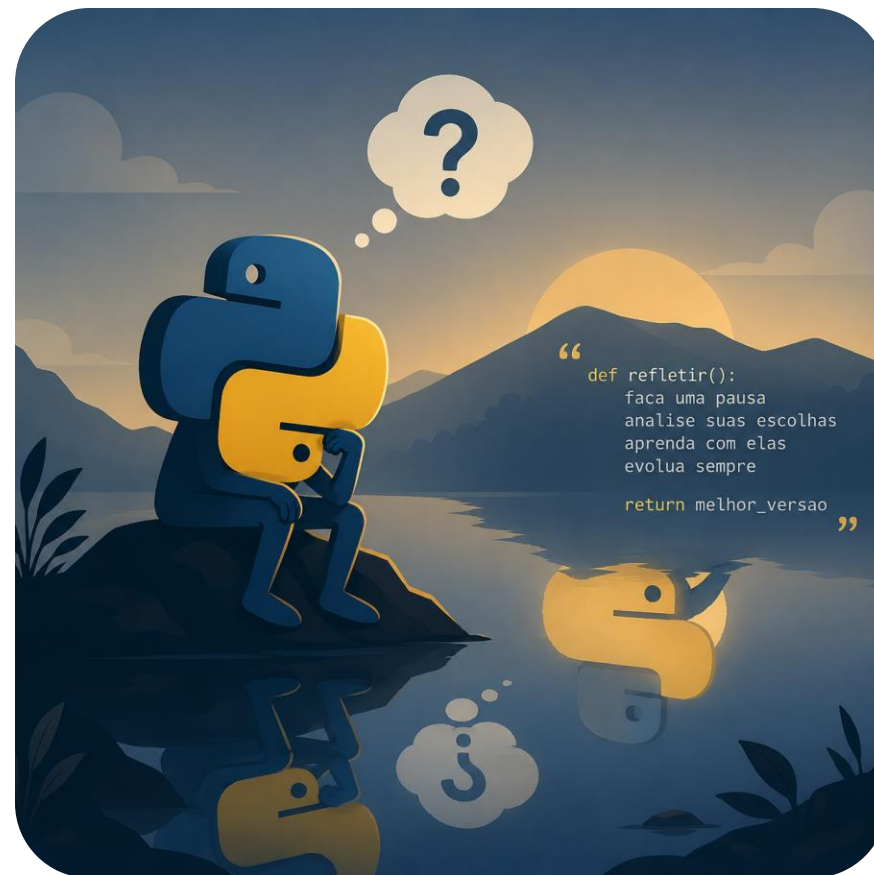
- nome
- idade
- matrícula

Essas informações pertencem ao aluno.



Método `__init__`

O `__init__` é executado automaticamente quando o objeto é criado.



Exemplo

Python

```
class Aluno:

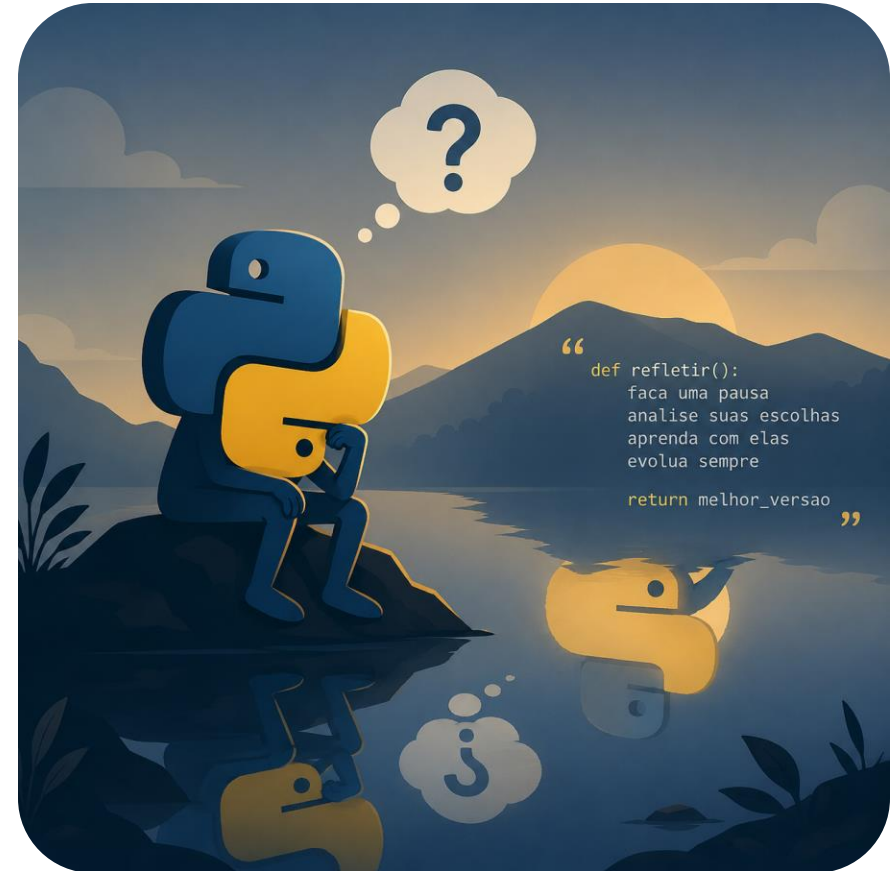
    def __init__(self, nome, idade):

        self.nome = nome
        self.idade = idade
```

Criando Objeto

Python

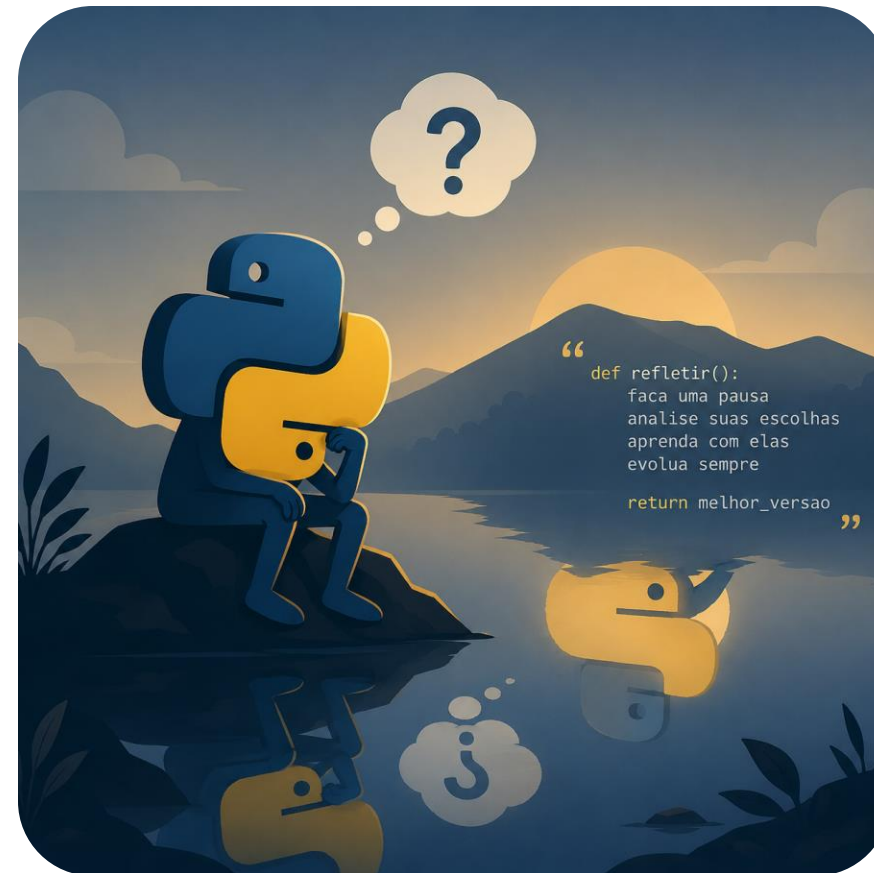
```
aluno1 = Aluno("João", 18)
```



Acessando atributos

Python

```
print(aluno1.nome)  
print(aluno1.idade)
```

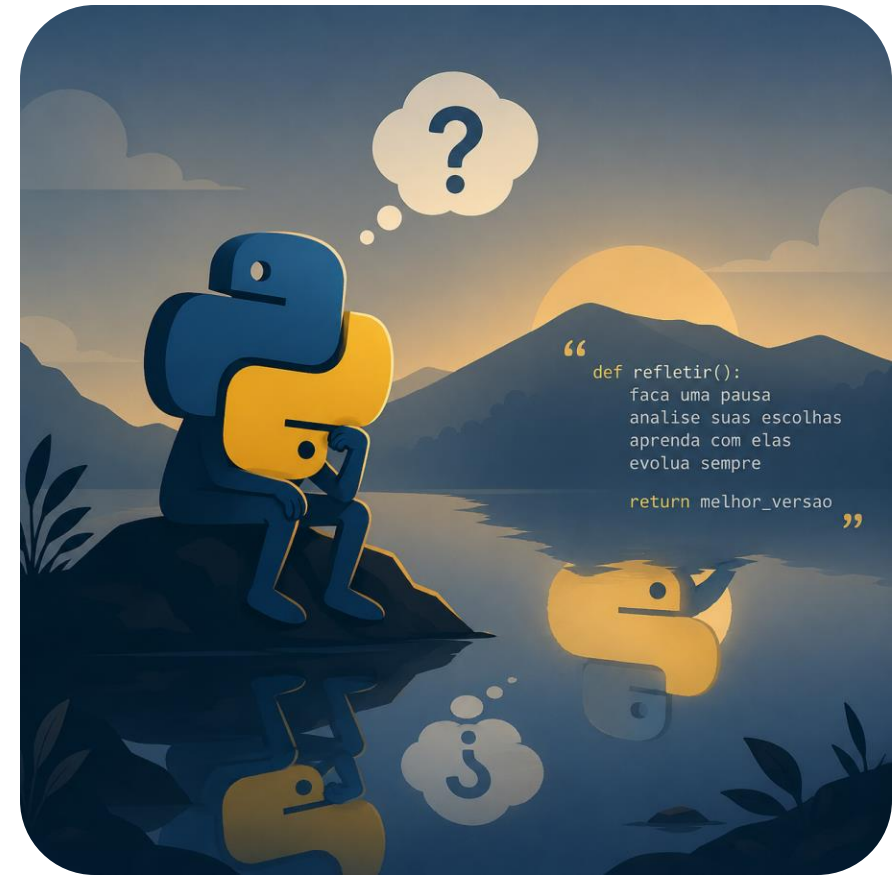


Reflexão Importante

O `self.nome` significa:

“o nome desse objeto específico”

Cada objeto possui seus próprios dados.



Three small purple squares arranged in a 2x2 grid pattern, with the bottom-right square missing.

Novo desafio

Three small purple squares arranged in a 2x2 grid pattern, with the bottom-right square missing.

Desafios

1. Classe `Produto`
2. Classe `Livro`
3. Classe `Filme`

Todos deverão possuir:

- nome
- preço ou categoria



Desafio

Criar uma classe `ContaBancaria`.

Ela deverá:

- armazenar saldo
- realizar depósito

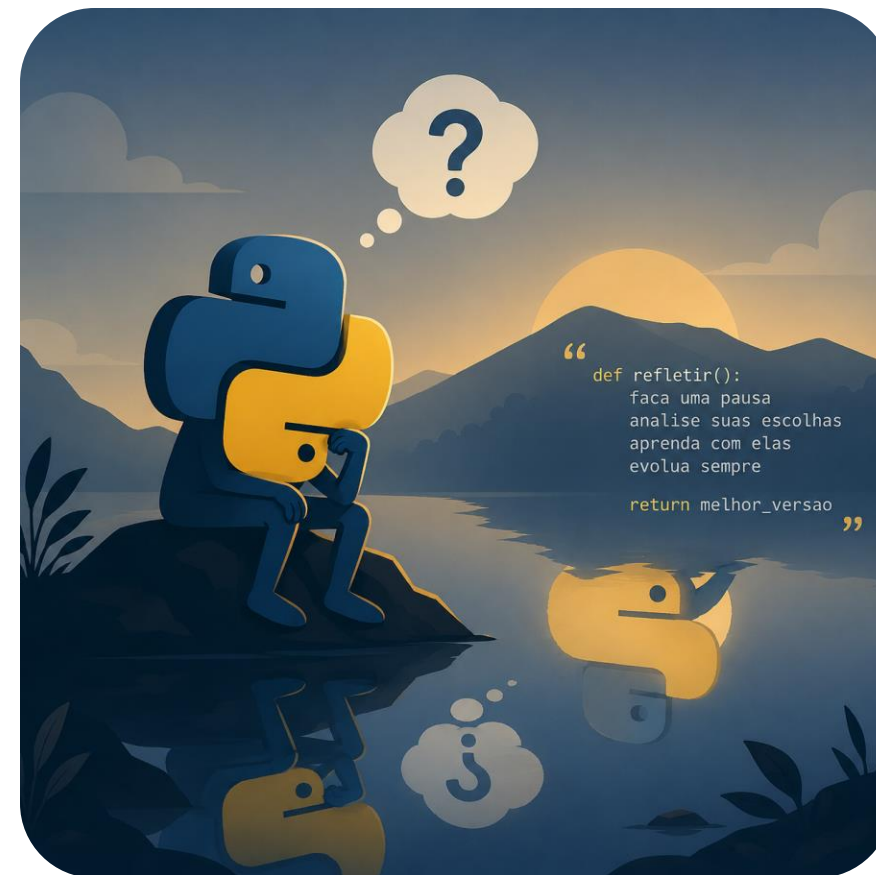




Reflexão

O que é um método?

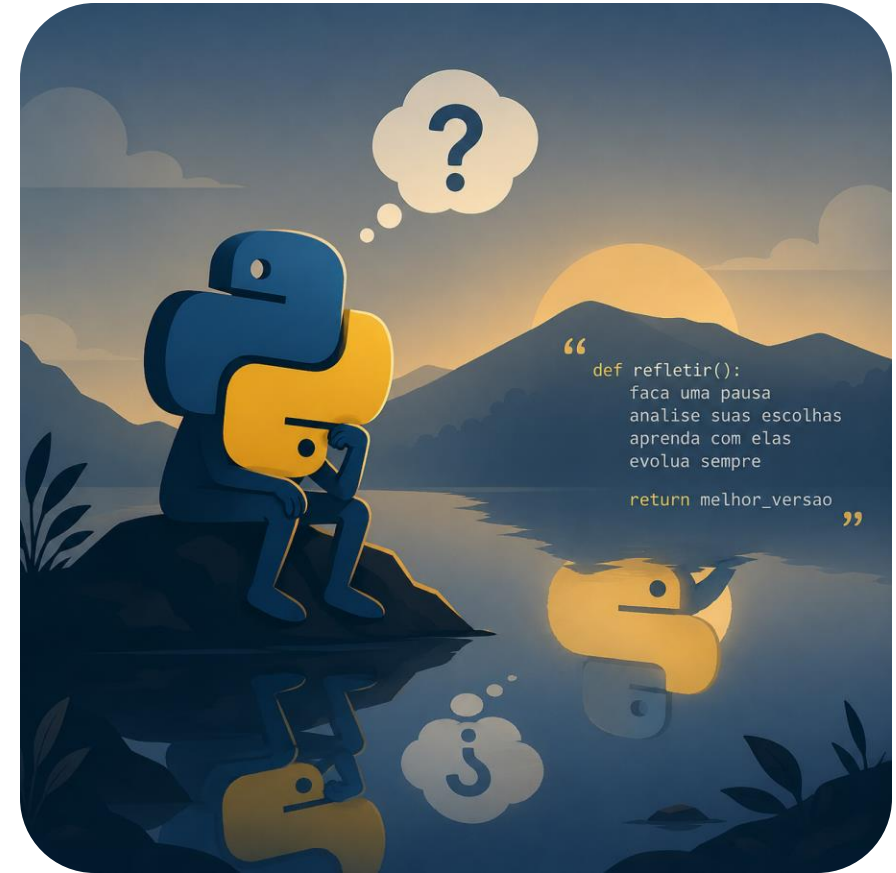
Método é uma função dentro da classe.



Comparação com Vida Real

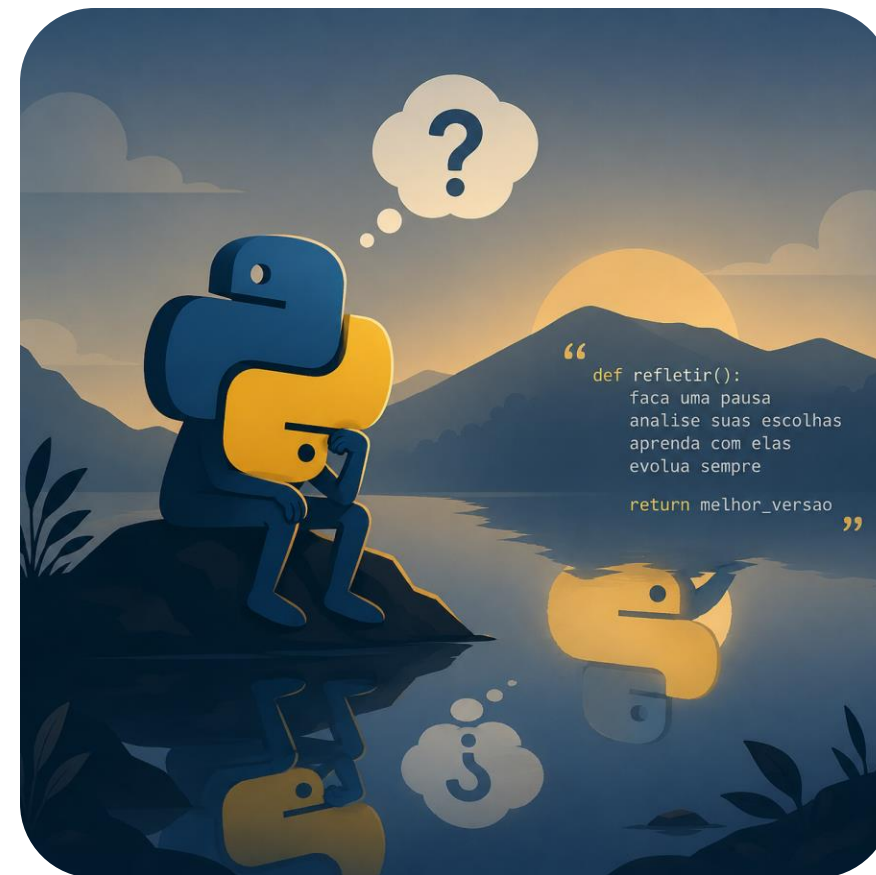
Conta bancária:

- possui saldo
- pode depositar
- pode sacar



Aplicativos bancários fazem exatamente isso:

- cada conta é um objeto
- cada cliente possui dados próprios
- os métodos modificam informações





Mais desafios? Simm



Desafios

1. Classe `Carrinho`
2. Classe `Jogador`
3. Classe `Estoque`

Cada classe deverá possuir:

- atributos
- métodos de alteração



`</>` Python

```
class Jogador:
```

```
    def __init__(self, nome):
```

```
        self.nome = nome
```

```
        self.pontos = 0
```

```
    def ganhar_pontos(self):
```

```
        self.pontos += 10
```



Sistema Completo com POO

Tempo: 20 min

Situação

Criar um sistema simples de loja utilizando POO.

Requisitos

Criar uma classe `Produto` com:

- nome
- preço

Criar métodos para:

- mostrar produto
- alterar preço

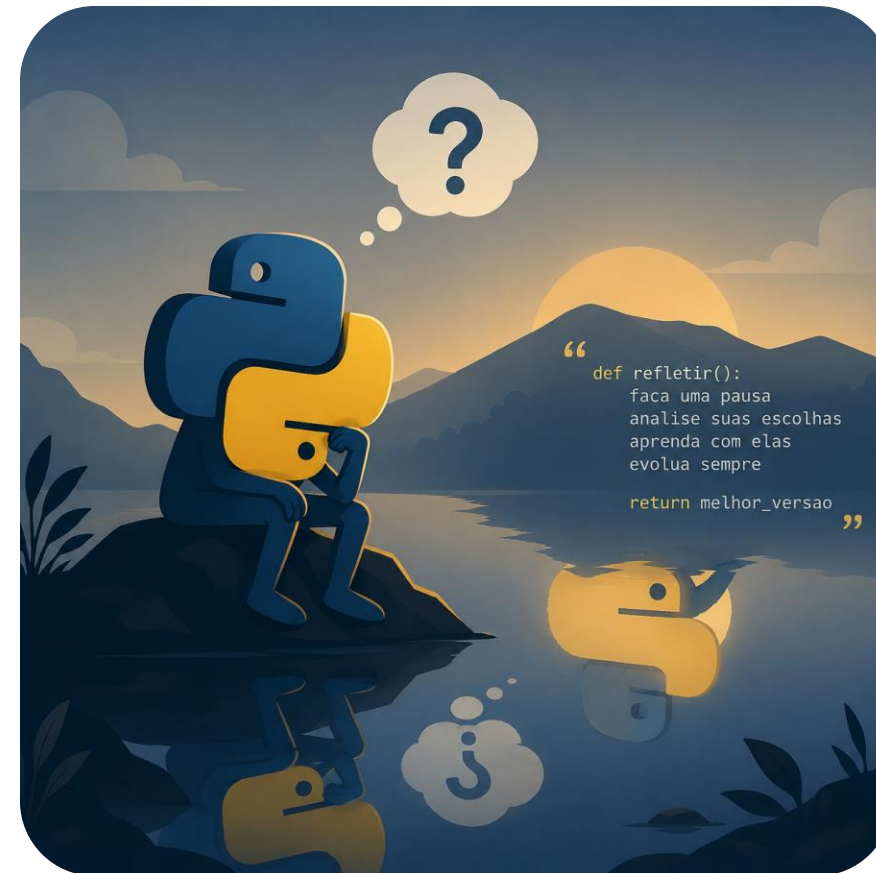




Reflexão

Perguntas para turma

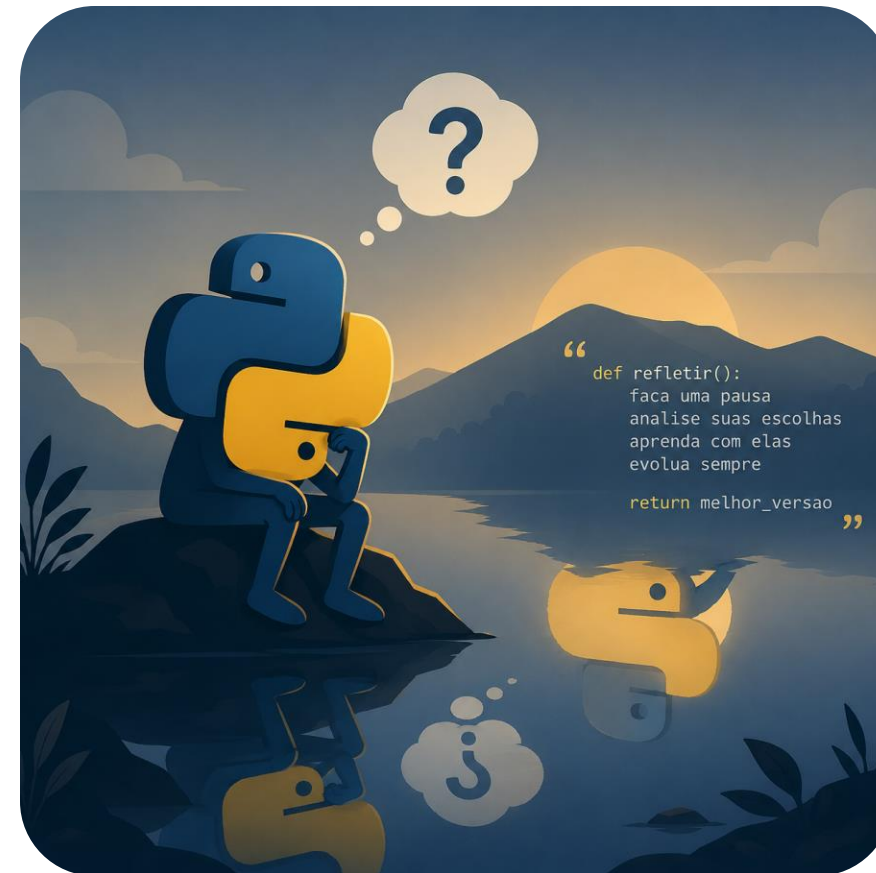
- Onde usamos objetos na vida real?
- Por que sistemas grandes utilizam POO?
- Como a organização ajuda no desenvolvimento?



Reflexão

Perguntas para turma

- Onde usamos objetos na vida real?
- Por que sistemas grandes utilizam POO?
- Como a organização ajuda no desenvolvimento?



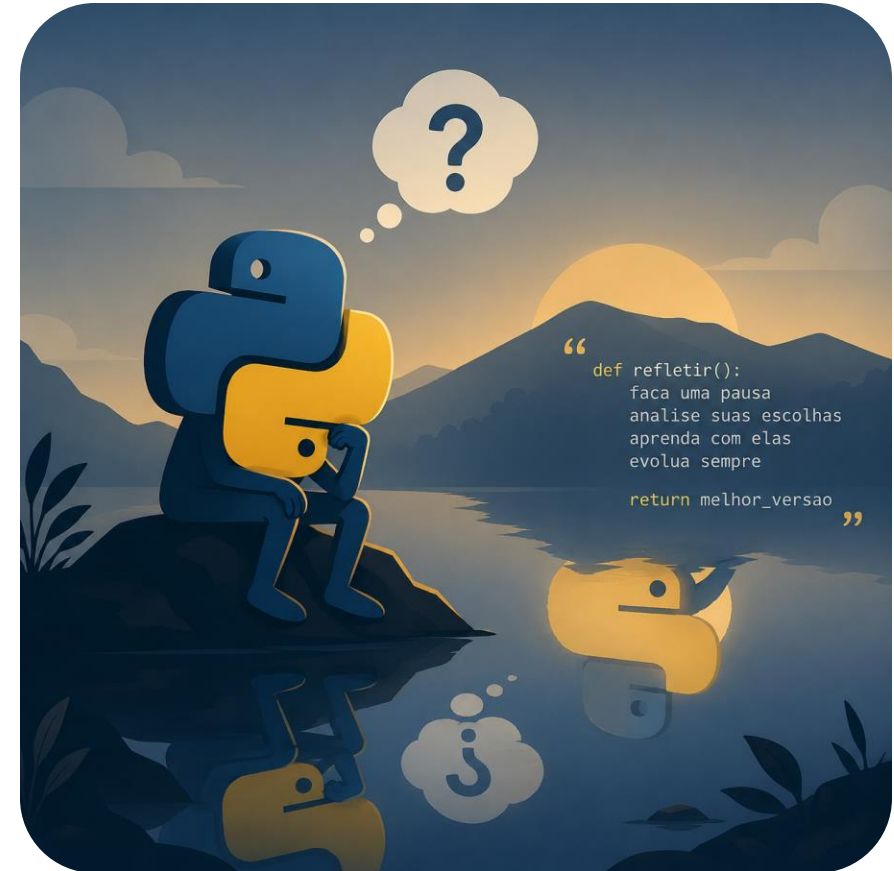
Grandes sistemas como:

- bancos
- jogos
- Instagram
- Uber
- Netflix

utilizam Programação Orientada a Objetos.

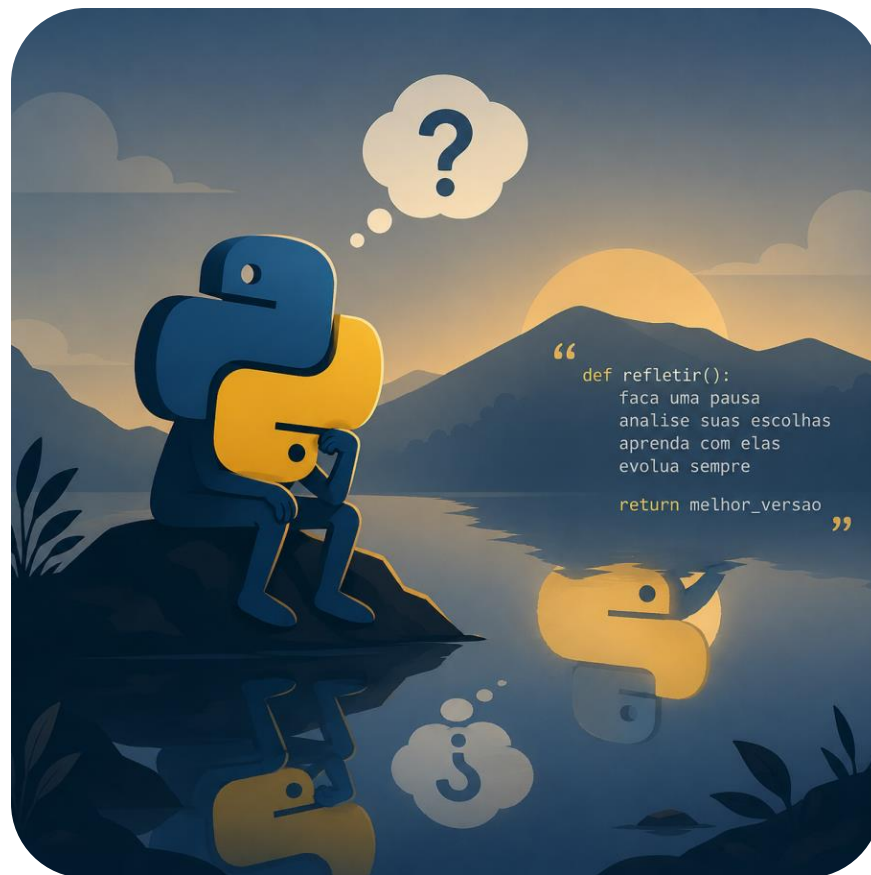
Cada objeto representa algo do mundo real:

- usuário
- carro
- conta
- produto
- pedido



RESUMO DOS CONCEITOS

Conceito	Explicação
classe	molde
objeto	item criado
atributo	característica
método	ação
<code>self</code>	próprio objeto
<code>__init__</code>	construtor



“A Programação Orientada a Objetos organiza o sistema como o mundo real: objetos possuem características e comportamentos.”

ATÉ A
PRÓXIMA
AULA!